FabOS: Hooking up Container Platforms with Time-Sensitive Networks

Manuel Eppler, Jochen Schenk, Tobias Gruner, Andy Zirkler, Harald Mueller, Andreas Blenk
{manuel.eppler,jochen.schenk,tobias.gruner,andreas.zirkler,h.mueller,andreas.blenk}@siemens.com
Siemens AG, Munich, Germany

ABSTRACT

There is no holistic approach that provides a full implementation and integration of flexible container platforms with time-sensitive networks in industrial factories yet. This hinders innovation and also withholds the advantages of flexible infrastructure from industrial network operators — a clear miss of optimization opportunities. To fill this gap, this paper proposes the FabOS architecture: a Kubernetes and Time-Sensitive-Networking-based architecture. Hence, FabOS provides flexible processing and deterministic networking for future industrial factories. With FabOS, use case developers can for the first time specify and implement applications that are deployed on Kubernetes connected via a time-sensitive network. Measurements taken from a proof-of-concept implementation demonstrate that future factories can benefit from bringing together container platforms and real-time networking.

CCS CONCEPTS

• **Computer systems organization** → **Real-time system architecture**; • **Networks** → *Link-layer protocols*; **Network reliability**;

KEYWORDS

Industrial Networking, Architecture, Time-Sensitive Networking, Kubernetes, Network Controller

ACM Reference Format:
Manuel Eppler, Jochen Schenk, Tobias Gruner, Andy Zirkler, Harald Mueller, Andreas Blenk. 2023. FabOS: Hooking up Container Platforms with Time-Sensitive Networks. In *Enhanced network techniques and technologies for the Industrial IoT to Cloud continuum (IIoT-NETs '23), September 10, 2023, New York, NY, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3609389.3610568

1 INTRODUCTION

The Context: Factories of the future are confronted with increasing challenging demands: adaptive production, sustainability, security, compute efficiency, and real-time networking requirements. For instance, the concept of adaptive production demands the factory to even adapt the floor plan potentially on a daily or even hourly basis. Consequently, the networking setup between machines and the operation systems of the factories (e.g., Machine Execution Systems) might change on a daily basis in the future. Beside this, new trends such as virtual programmable logic controllers enable the flexible use of common of-the-shelf computing hardware. To not reinvent the wheel and to fully utilize the available resources, concepts such as Kubernetes (K8s) are envisioned to also be used in future factories.

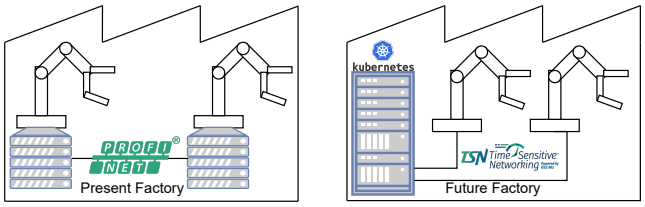


Figure 1: Comparison of the present and future factory design.

The Problem: In the IT world and, in particular, the cloud world, container platforms such as K8s have led to a breakthrough w.r.t flexible resource usage. Container platforms are the de-facto standard in IT environments where flexible processing and service provisioning is of uttermost importance. Whereas container platforms are designed and configured in particular for IT environments, they have not yet been adapted to industrial networking scenarios. Furthermore, when looking at the industrial networking technologies, Time-Sensitive Networking (TSN) is a promising solution to increase flexibility of the network landscape in factories. TSN is seen as an open successor of prioprietary protocols such as ModbusTPC to realize the Quality-of-Service (QoS) as required by industrial applications. While the integration of TSN in industrial networks is under heavy investigation, the combination of both container platforms and TSN is still at its beginning. This paper takes the next step and proposes the FabOS architecture to fill this gap.

The Solution - FabOS Seamless Computing Cluster: The FabOS Seamless Compute Cluster (FSCC) combines both worlds: K8s and Time-Sensitive Networking (TSN). Fig. 1 provides a high-level comparison of the traditional factory setting and the contribution of this paper, the FSCC. In particular, FSCC does not only bring K8s and TSN together, it also addresses all important aspects of industrial networks. First, it provides a full Kubernetes-based configuration and demand request capability. Second, it addresses the security setups of existing factory concepts, such as the division into different security zones, by introducing the network harmonizer setting.

This work is licensed under a Creative Commons Attribution International 4.0 License.

IIoT-NETs '23, September 10, 2023, New York, NY, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to the Association for Computing Machinery.
ACM ISBN 979-8-4007-0302-7/23/09
https://doi.org/10.1145/3609389.3610568

Third, it enables the flexible use of time-sensitive networking mechanisms such as IEEE 802.1Qci Per-Stream-Filtering-And-Policing (PSFP) [12].

The Contribution: Besides presenting the high-level architecture of the FSCC, this paper presents a full proof-of-concept implementation of FSCC. Measurement results for different network traffic scenarios show the working behavior of FSCC.

This work does not raise any ethical issues.

2 SCENARIO AND PROBLEM STATEMENT

This section presents the current production plant networking and problems that come with them. Afterwards, it introduces the basics of Time-Sensitive Networking (TSN) and networking in K8s. Lastly, the section declares the upcoming problems.

2.1 Production Plant Networking

A typical production plant is clustered into multiple production cells, that for example consist of multiple robots, bolting some parts together. Afterwards, the combined part is moved to the next cell, where further parts are added. Production cells from multiple vendors are combined to a production line, whereas a production plant consists of multiple production lines. Before integrating the production cell into the production line or plant, it needs to be tested. Therefore, each production cell comes with a preconfigured network with its own Quality of Service (QoS) requirements. Connecting those production cells and combining them to a single production plant network is a tedious work, since IP address ranges of multiple production cells might overlap or even be identical. Existing solutions use routers and IPv6 address delegation to change the IP addresses within a production cell to start with a special prefix. Although this solves the addressing of services inside production cells, many production cell components do not support IPv6 or IPv4 routing, e.g., TSN network controllers need data link layer (l2) access to TSN capable bridges.

2.2 Time-Sensitive Networking (TSN)

Time-Sensitive Networking (TSN) is an enhancement for Ethernet networks standardized by the Institute of Electrical and Electronics Engineers (IEEE) 802.1 TSN task group [3]. It introduces QoS guarantees by policing and shaping of periodic data transmissions, called streams. Examples for those QoS guarantees are bounded low latency, high reliability and bounded maximum jitter. Streams consist of one or multiple data frames that are sent periodically from an end station, called talker, through multiple bridges to one or multiple end stations, called Listeners. In the fully centralized network model, end stations send their stream information, containing QoS requirements, to a centralized component, called centralized user configuration (CUC). Afterwards, the CUC sends the collected data to the centralized network configuration (CNC) that configures TSN components, e. g. policing and shaping within bridges, such that all QoS requirements are fulfilled.

Traffic Scheduling: TSN capable bridges utilize multiple queues to implement traffic scheduling. Those queues can only forward frames according to the first in first out (FIFO) characteristic which means that frames within the same queue are sent in the same order as received. Due to this characteristic, TSN capable bridges enable reordering of frames by using 8 prioritized queues. The frames are assigned to the queues according to the priority code point (PCP) of the virtual local area network (VLAN) header as long as not overwritten, e.g., by Per-stream Filtering and Policing (PSFP). Based on this forwarding mechanism, TSN standardizes multiple traffic scheduling and shaping algorithms, e.g., strict priority scheduling, credit based shaper, and time aware shaping.

Strict Priority traffic scheduling is the standard scheduling mechanism within ethernet bridges. The PCP value inside the VLAN header of a frame assigns it to one priority queue of the bridge. An en-queued frame has to wait until there are no more frames within the queues of higher priority and it is the oldest frame within its own queue. TSN gives latency bounds for this scheduling mechanism by restricting the data rate of frames with high priorities using PSFP.

Per-stream Filtering and Policing (PSFP) implements stream filtering and policing prior to the priority queues. It consists of three layers called stream identifiers, stream gates and flow meters. The stream identifiers assign frames to a stream gate and a flow meter based on the frames source or destination address, the VLAN ID and the VLAN PCP. Stream gates can drop frames based on a periodic schedule. In addition, they can assign an internal priority value to frames which overrides the priority queue assignment within this bridge. Flow meters are two rate three color metering instances restricting the data-rate of streams before the frames are enqueued.

2.3 Kubernetes (K8s)

K8s is an open source system for a scalable deployment and management of container applications [5]. The containers run as pods on compute nodes that are managed by the K8s master. Furthermore, the K8s master is built on a modular basis with multiple application programming interfaces (APIs) to extend its functionality, e.g., to implement custom resource allocations and network interfaces. In general the resource allocation is called scheduling and based on resource requirements like CPU and RAM usage, hardware, software and policy constraints as well as configured affinities. Although resource requirements and hardware constraints are taken into account, network resources are not. Speaking of networking, K8s uses a network layer (l3) overlay network, which means that each node has its own IP subnetwork assigned for its pods. Each pod has a virtual network interface with an assigned IP address, that is connected to the outside via a virtual network bridge on its host. The network bridge allows communication between all containers on this host and tunnels the traffic to other nodes of the K8s cluster. Although by default each pod uses one network interface for all its communication, the Multus plugin defines a container network interface (CNI) other plugins can use to implement l3 interfaces [10]. Examples for those plugins are Calico and Weave [4, 7]. However, none of those plugins implements real time communication support.

2.4 Problem Statement

This section states four problems solved within this paper:

First, a component is needed that connects multiple production cells with l2 access and high reliability standards, while introducing low configuration overhead, to connect the clustered production plant networks. Section 3.1 introduces the Network Harmonizer (NH) as a solution for easy connection of multiple production cells.
Second, the problem is the connection between K8s pods through a real time capable TSN network. Therefore, each real time capable pod needs access to TSN capabilities, like adding Ethernet VLAN tags to frames, which are l2 features that are lost by routing. The Kubernetes TSN Networking paragraph in section 3.3 refers to this problem and states a solution.
Third, K8s needs an interface to communicate with the TSN network to request streams and allocate network resources for them. The Seamless Compute Network Service (SCNS), introduced in section 3.3, is the implementation of such an interface on the side of K8s, to the Modular Network Controller (MoNC) on the network side.
Fourth, the resource allocation process of K8s needs an enhancement, called network aware scheduling, to take current network capabilities like available data-rate and possible latency guarantees into account. The network aware scheduling is implemented by the QoS scheduler in section 3.3.

3 FABOS SEAMLESS COMPUTE CLUSTER

This section introduces the FabOS seamless compute cluster, a proof of concept implementation that brings the flexibility of cloud computing to production networks. The first section addresses the NH that connects off-the-shelf production cells while enabling l2 access from production plant to production cell components. Afterwards, the MoNC, a controller for TSN networks is introduced while section 3.3 states the implementations needed to enhance Kubernetes with TSN capabilities. Finally, the application deployment process states the interactions between the cluster components during the deployment of an application.

3.1 Network Harmonizer (NH)

The NH is a new network component that connects a production cell network with the production plant network. Similar to a router, it transports packets into the other network if the destination address belongs to that network. In contrast to a router, the NH is fully transparent to the devices, i.e. they communicate as if they are within the same data link layer (l2) network. The NH substitutes the production plant IP addresses within passing packets to the production cell IP address and vice versa. This substitution applies to source- and destination-IP addresses as well as to sender- and target-IP addresses in ARP packets. With that, the production cell addresses do not need to be unique across multiple production cells, since production cells are only interconnected through NHs. However, the production plant address can not be reached inside the production cell network without being routed through the NH. Depending on the configuration of the NH, all or only selected addresses can be routed between network segments. Additionally, if the MAC addresses are not unique across production cells, which is common although violating the Ethernet standard, they are replaced in an analog manner.

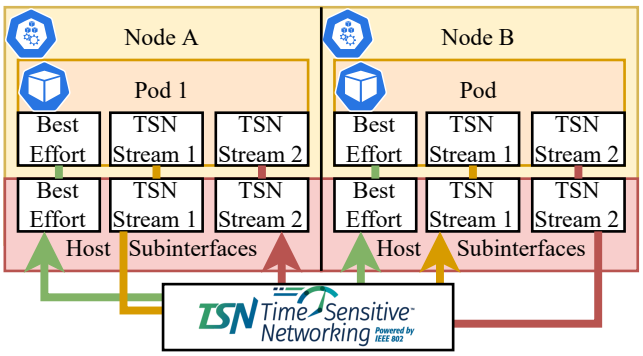


Figure 2: Network path between two Kubernetes pods according to the proposed CNI plugin.

3.2 Modular Network Controller (MoNC)

For configuring network components IEEE Std. 802.1Qcc defines, besides others, a fully centralized model that uses a centralized user configuration (CUC) to collect stream requirements and a centralized network configuration (CNC) that configures the network devices accordingly [13]. MoNC provides modular extensible interfaces to CUCs and network nodes. The CUC interface implements a Web, REST, and NetConf interface to collect stream requirements from different sources. Additionally, the CNC gathers topology information utilizing link layer discovery protocol (LLDP). Based on both the network topology and the requested streams from the CUC, the CNC calculates configuration parameters for the bridges and end stations such that the timing behavior of all streams is within the QoS guarantees. Examples for those configuration parameters are, in case of strict priority shaping with PSFP, the stream id, priority code point, committed information rate, and committed burst size. The CNC configures the bridges via ssh or NetConf protocol accordingly and returns the embedding status and the stream configuration, i.e. configured VLAN ID and PCP.

3.3 QoS Scheduler

The QoS scheduler is an extension for K8s that implements an extended model using custom resource definitions (CRDs) for describing network relations inside the cluster and applications and their QoS requirements to the network. The introduced CRDs are application groups, applications, assignment plans, compute nodes, network links, and streams. Multiple QoS-aware software components, called workloads, are combined to an application and multiple applications to an application group. All workloads of an application are scheduled together considering network requirements and resource availability of the cluster. Either all workloads and streams of an application are deployed or none of them. On application group level the system tries to place as much applications as possible. The network resources and statistics like link data-rate, unused data-rate, and estimated latency are modeled as network links, which are unidirectional links between compute nodes. From a Network perspective, this implements the centralized user configuration (CUC) functionality. The following sections introduce the QoS scheduler implementation with its components

Seamless Compute Controller (SCC), Seamless Compute Optimizer (SCO), Seamless Compute Scheduler (SCS), and Seamless Compute Network Service (SCNS) and their interfaces.

The Seamless Compute Controller (SCC) is the core of the QoS Scheduler. It coordinates custom resources and the resource allocation process by using the K8s controller-runtime to receive messages from the Kubernetes system, e.g. applications, pods or streams. Additionally, the Seamless Compute Controller (SCC) uses REST APIs for interfacing components, that do not yet support CRDs, like the MoNC and the Seamless Compute Optimizer (SCO).

The Seamless Compute Optimizer (SCO) receives resource requests via a REST API. Afterwards, the resource allocation is computed and the result is asynchronously delivered to the SCC as a CRD, called assignment plan. This plan is then used by the Seamless Compute Scheduler to assign workloads to compute nodes. The SCO also serializes requests to avoid over-provisioning by assigning the same resources multiple times in different requests.

The Seamless Compute Scheduler (SCS) implements a custom scheduler using the plugin system of the K8s scheduler framework [6]. It uses the previously generated assignment plan CRD to get information which pod to bind to which cluster node. First, the nodes are scored for the placement of the pods using the score plugin. After applications are allocated by the SCC the K8s scheduler verifies whether the entry in the assignment plan corresponds to one of the eligible nodes. The plugin ensures that the preferred node receives the highest score. In the last step the permit plugin starts the pods as soon as their streams are embedded by the MoNC.

The Seamless Compute Network Service (SCNS) is the interface to MoNC. It periodically receives the current network topology and requests TSN streams for each interface between workloads that need to be scheduled. According to the responses of the MoNC, the network service updates the CRDs for streams and network links.

Kubernetes TSN Networking. The Multus plugin enables multiple network interfaces within the same K8s pod [10]. In particular, standard traffic, called best effort traffic, and all TSN streams get their own network interface such that pods can use TSN streams based on the interface's Internet Protocol (IP) address. For ease of use, K8s services can simply be accessed from within a pod by their domain name service (DNS) name. Therefore the static DNS name gets translated to a dynamic IP address of the interfaces. That way applications do not need any TSN specific implementation but only use the DNS-name of the destination service.

As shown in Figure 2, the main interface of the pod is used for standard, non TSN traffic like K8s cluster management traffic. The shown setup uses the cluster's default CNI plugin for this interface. Other interfaces are controlled by a custom container network interface (CNI) plugin that receives the TSN stream on a virtual interface and forwards it via a virtual bridge to a sub-interface of the node's physical interface. This sub interface adds the required VLAN header with VLAN ID and VLAN PCP for seamless TSN communication.

3.4 Application Deployment Process

First, MoNC learns the topology of the network and the Seamless Compute Network Service registers for topology updates from MoNC. The network service stores the topology into the according CRDs. If a user requests the deployment of applications and application groups, the SCC combines information on the requested application with information on the current topology and calls the SCO. The assignment of workloads to nodes is then returned as an assignment plan, also containing the required network streams. Afterwards, SCNS requests the required network streams from the MoNC which checks, if the network is able to guarantee those streams, e.g. by calculating the worst case latencies. It either embeds the streams on the bridges or returns an error, which leads to a new assignment optimization. Lastly, the Seamless Compute Scheduler (SCS) places the pods according to the assignments within the assignment plan.

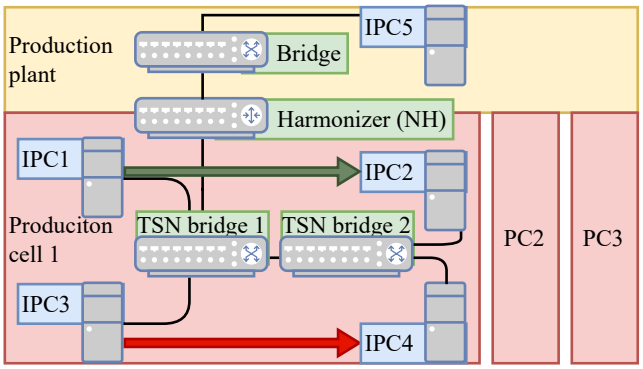


Figure 3: Topology of the FabOS test network [8].

4 EVALUATION

This section presents the measurement results of the SCC. First, the test setup is explained, second, the measurement results are shown by presenting SCC protection capabilities and the inserted overhead of Kubernetes.

4.1 Test Network

Components: The test network, shown in Fig. 3, consists of 6 industrial PCs (IPCs) from Siemens [2], 2 TSN capable bridges from NXP [1], and 1 Ethernet bridge. Besides the up-link, the network is split into two zones that are connected with an IPC which implements the NH functionalities. The production plant network, also called yellow zone, contains the IPC5 connected to the Ethernet bridge that has no TSN capabilities. The production cell, called red zone, contains two TSN capable bridges and the IPC 1 to IPC 4. IPC 1 and IPC 2 are equipped with a Intel I210 network card to enable sub microsecond time synchronisation between them using precision timing protocol (PTP) [14], which is needed for precise measurement results. Software-wise, the IPCs use Ubuntu 20.04 as operating system and they are connected by a single Kubernetes (K8s) cluster with IPC5 as K8s master.

Procedure: For the measurements, prioritized traffic is sent from IPC 1 and IPC 3 to IPC 2 and IPC 4. Traffic from IPC 1 to IPC 2, shown as green arrow, is used for the measurements while traffic from IPC 3 to IPC 4, shown as red arrow, is used to interfere the

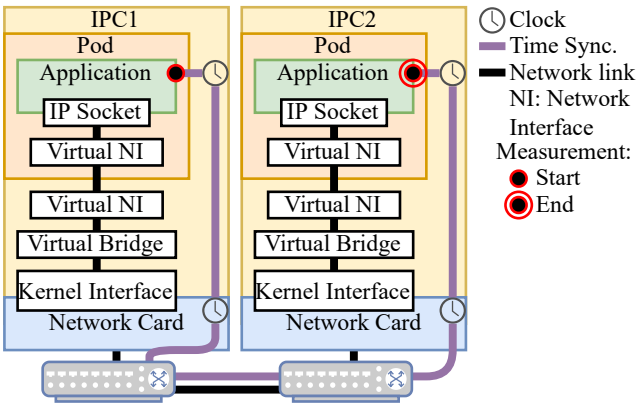


Figure 4: End-to-end measurement points of the experiment setting with Kubernetes (K8s).

connection. The link between the TSN capable bridges becomes the connections bottle neck.

Applications A client-server application written in C, called TSN test app, measures the end-to-end latency by sending Ethernet frames from the client to the server application. The TSN test app can either run native within Linux, or run as container within a K8s pod. The sent frames contain an ID, a sending timestamp as well as some random data to reach the requested frame size. The server application receives the frames and calculates the latency. A slightly modified version of the TSN test app sends interference traffic with a precise frequency.

4.2 Measurements

The first measurements show that the implementation is capable of protecting network traffic between K8s workloads against high priority interference traffic. The other measurements analyze the overhead of the proposed system. It investigates the latency that Kubernetes (K8s) and the CNI plugin introduce.

4.2.1 Traffic protection. To show effects of traffic protection, 1 kilobyte sized measurement frames are sent with a frequency of 10 kilohertz from IPC 1 to IPC 2 which results in a data-rate of 80 megabit per second. Those frames are either sent without PSFP based protection on priority 3 or with protection on priority 5. The first two measurements record the baseline without any high priority interference traffic. To show the protection in action, the measurements three and four introduce interference traffic from IPC 3 to IPC 4. IPC 3 sends 1 kilobyte sized frames with a frequency of 100 kilohertz which results in a data-rate of 800 megabit per second. For both, the protected and unprotected case, the interference traffic is sent with the same priority, as the measurement traffic.

Results: Fig. 5 shows the traffic protection capabilities of the SCC. It compares the empirical cumulative distribution function (CDF) for the end-to-end latency of the four runs. The baseline runs with and without protection as well as the protected run with interference traffic are very close to each other whereas the unprotected run with interference is clearly moved to the right. This means that the interference traffic does indeed lead to additional delay of the

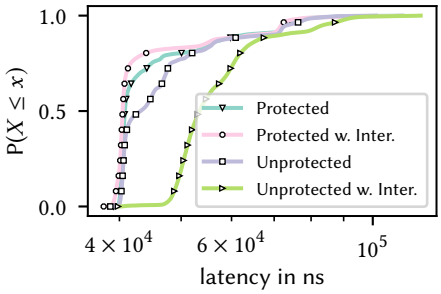


Figure 5: Latency comparison of protected and unprotected streams with and without interference traffic.

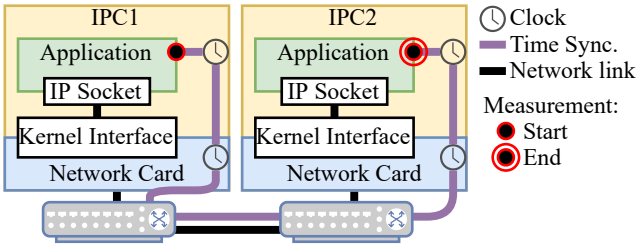


Figure 6: End-to-end measurement points for native network layer (l3) application.

unprotected run whereas the QoS deployment does protect the streams against interference traffic.

4.2.2 Overhead results. This section compares the latency of native and K8s deployments to visualize the latency overhead of the proposed setup. To measure the latency overhead introduced by the K8s setup, multiple runs of sending the same measurement traffic with different deployments are compared. The latencies are measured twice during the same run. Once by recording pcap files using tcpdump on both endpoints, and once using the TSN test app. The measurement traffic consists of 1 kilobyte sized frames that are sent with a frequency of 10 kilohertz from IPC 1 through both bridges to IPC 2. First the TSN test app runs as native deployment using the Linux IP socket as shown in Fig. 6. That way, the measurement does not include the Kubernetes overhead.

Results: Fig. 7 contains the latency for deployment variants and measurement points to show where the latency stems from. Since the pcap files of all three deployments have almost identical latency distributions, only one of them is shown within the figure. With a mean delay of around 33μs most of the delay stems from the network connection between the Linux kernels. On average, another 11μs delay is introduced by the TSN test app. Additionally, 20% of the highest latencies experience an average delay overhead of up to 30μs. In comparison to the native deployment, the proposed setup only adds another 1.5μs average delay. For larger networks it is expected that the delay within the end-host remains identical whereas the delay from the network is increased based on the hop count and with it the difference between protected and unprotected traffic

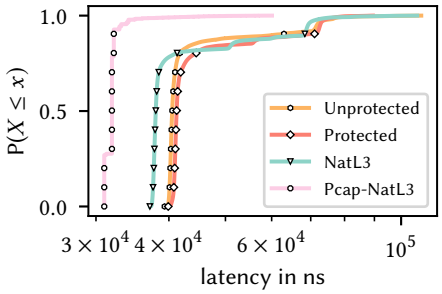


Figure 7: Latency comparison between native and K8s TSN test app deployment.

5 RELATED WORK

Previous research on virtualisation of critical applications investigated the orchestration of computing resources and CPU scheduling [11, 16]. None of those works consider network resources like TSN capabilities. Garbugli et al. propose with KuberneTSN a K8s CNI plugin that enables deterministic network access for containers [9]. Although they state, that container scheduling processes are needed, they do not introduce such a resource scheduling process. Other research was focused on accelerating inter container networking, e.g. by removing multiple kernel processing runs for frames [17] or by kernel-bypassing [15]. All those are not compatible with TSN and do not contain any network aware scheduling.

6 CONCLUSION

The FabOS Seamless Computing Cluster provides an industrial network architecture concept that combines Kubernetes with time-sensitive networking. Measurements taken from a proof-of-concept implementation show that applications running on Kubernetes perceive QoS guarantees using TSN technology. The introduced overhead of Kubernetes is still acceptable, while applications perceive low end-to-end latency. We believe that our paper opens interesting directions for future work to integrate real-time capabilities into Kubernetes networking: real-time monitoring, usage of other traffic shaping mechanisms, and redundancy mechanisms.

REFERENCES

[1] 2023. Layerscape LS1028A Reference Design Board. (2023). https://www.nxp.com/design/qoriq-developer-resources/layerscape-ls1028a-reference-design-board:LS1028ARDB
[2] 2023. SIMATIC Rack IPC. (2023). https://www.siemens.com/de/de/produkte/automatisierung/pc-based/simatic-rack-ipc.html
[3] IEEE 802.1. 2023. Time-Sensitive Networking (TSN) Task Groups. (2023). https://1.ieee802.org/tsn/
[4] Projectcalico authors. 2023. Calico. (2023). https://github.com/projectcalico/calico
[5] The Kubernetes authors. 2023. Kubernetes. (2023). https://kubernetes.io/
[6] The Kubernetes authors. 2023. Scheduling Framework. (2023). https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/
[7] Weaveworks authors. 2023. Weave Net - Weaving containers into applications. (2023). https://github.com/weaveworks/weave
[8] FabOS Consortium. 2023. FabOS. (2023). https://www.fab-os.org/
[9] Andrea Garbugli, Lorenzo Rosa, Armir Bujari, and Luca Foschini. 2023. KuberneTSN: a Deterministic Overlay Network for Time-Sensitive Containerized Environments. *arXiv preprint arXiv:2302.08398* (2023).
[10] Kubernetes Network Plumbing Working Group. 2023. Multus CNI. (2023). https://github.com/k8snetworkplumbingwg/multus-cni
[11] János Harmatos and Markosz Maliosz. 2021. Architecture Integration of 5G Networks and Time-Sensitive Networking with Edge Computing for Smart Manufacturing. *Electronics* 10, 24 (Dec 2021), 3085. https://doi.org/10.3390/electronics10243085
[12] IEEE. 2017. IEEE Standard for Local and metropolitan area networks–Bridges and Bridged Networks–Amendment 28: Per-Stream Filtering and Policing. *IEEE Std 802.1Qci-2017 (Amendment to IEEE Std 802.1Q-2014 as amended by IEEE Std 802.1Qca-2015, IEEE Std 802.1Qcd-2015, IEEE Std 802.1Q-2014/Cor 1-2015, IEEE Std 802.1Qbv-2015, IEEE Std 802.1Qbu-2016, and IEEE Std 802.1Qbz-2016)* (2017), 1–65. https://doi.org/10.1109/IEEESTD.2017.8064221
[13] IEEE. 2018. IEEE Standard for Local and Metropolitan Area Network–Bridges and Bridged Networks – Amendment: Stream Reservation Protocol (SRP) Enhancements and Performance Improvements. *IEEE Std 802.1Qcc-2018* (2018).
[14] IEEE. 2020. IEEE Standard for Local and Metropolitan Area Networks–Timing and Synchronization for Time-Sensitive Applications. *IEEE Std 802.1AS-2020 (Revision of IEEE Std 802.1AS-2011)* (2020), 1–421. https://doi.org/10.1109/IEEESTD.2020.9121845
[15] Bojie Li, Tianyi Cui, Zibo Wang, Wei Bai, and Lintao Zhang. 2019. SocksDirect: Datacenter sockets can be fast and compatible. In *Proceedings of the ACM Special Interest Group on Data Communication*. 90–103.
[16] László Toka. [n. d.]. Ultra-Reliable and Low-Latency Computing in the Edge with Kubernetes. 19, 3 ([n. d.]), 31. https://doi.org/10.1007/s10723-021-09573-z
[17] Danyang Zhuo, Kaiyuan Zhang, Yibo Zhu, Hongqiang Harry Liu, Matthew Rockett, Arvind Krishnamurthy, and Thomas Anderson. 2019. Slim:{OS} kernel support for a low-overhead container overlay network. In *16th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 19)*. 331–344.

Appendices are supporting material that has not been peer-reviewed.

A ACRONYMS

API application programming interface
CNC centralized network configuration
CNI container network interface
CRD custom resource definition
CUC centralized user configuration
DNS domain name service
FIFO first in first out
FSCC FabOS Seamless Compute Cluster
IEEE Institute of Electrical and Electronics Engineers
IP Internet Protocol
IPC industrial PC
K8s Kubernetes
l2 data link layer
l3 network layer
LLDP link layer discovery protocol
MoNC Modular Network Controller
NH Network Harmonizer
PCP priority code point
PSFP Per-stream Filtering and Policing
PTP precision timing protocol
QoS Quality of Service
SCC Seamless Compute Controller
SCNS Seamless Compute Network Service
SCO Seamless Compute Optimizer
SCS Seamless Compute Scheduler
TSN Time-Sensitive Networking
VLAN virtual local area network